

SSH Configuration, Key Management & Tmux Multiplexing

Secure Shell (SSH) is the bedrock of modern infrastructure. It is not just a terminal emulator; it is a complex cryptographic tunnel capable of routing X11 graphics, local ports, and UNIX sockets across hostile networks. A Staff engineer must understand the underlying math, the operating system daemon architecture, and the catastrophic vulnerabilities introduced by poor key management.

7.0 The SSH Protocol & Cryptographic Handshake

When you type `ssh user@server`, the ensuing connection is not instantaneous. It requires a highly orchestrated, multi-level mathematical handshake designed to establish absolute trust over an untrusted medium (the Internet) where anyone could be wiretapping.

The 5 Phases of the SSH Connection

1. **The TCP Handshake:** Standard Layer 4 SYN, SYN-ACK, ACK.
2. **Version Exchange:** Both sides send plaintext strings (e.g., `SSH-2.0-OpenSSH_8.9p1`) to negotiate protocol compatibility.
3. **Key Exchange (KEX):** The most CPU-intensive phase. The client and server agree on a temporary, session-specific encryption key using Diffie-Hellman math. *No authentication has happened yet.*
4. **Host Authentication:** The server proves its identity to the client to prevent Man-in-the-Middle (MITM) attacks.
5. **User Authentication:** Finally, the client proves their identity to the server (via Passwords or Public Keys).

The Diffie-Hellman Miracle (KEX Math)

How do a client and server agree on a secret key over a public network where an attacker is listening to every packet? They use **Diffie-Hellman (DH) Key Exchange**.

The Paint Analogy: Client and Server agree publicly on a common color (Yellow). The Client picks a secret color (Red), mixes it with Yellow to make Orange, and sends Orange over the network. The Server picks a secret color (Blue), mixes it to make Green, and sends Green. The Client takes the Server's Green, adds its secret Red, resulting in Brown. The Server takes the Client's Orange, adds its secret Blue, resulting in Brown. *They now both possess the exact same Brown key, but Brown was never transmitted over the network.*

The Math: Instead of paint, DH uses modular arithmetic with massive prime numbers: $(g^a \bmod p)$. Even if a hacker intercepts the public "mixed colors", figuring out the original secret keys requires calculating discrete logarithms—a problem modern computers cannot solve in a billion years.

The Transition: Asymmetric to Symmetric Cryptography

Asymmetric math (using Public/Private key pairs) is brilliant, but it is astronomically slow. You cannot use RSA to encrypt a 1GB database dump over SSH; it would take hours. Therefore, SSH only uses Asymmetric Cryptography to *authenticate* and perform the DH Key Exchange. Once the shared "Brown" key is generated, SSH instantly abandons asymmetric math and switches to **Symmetric Cryptography** (like AES-256-GCM or ChaCha20). Symmetric encryption uses the exact same key to lock and unlock the data, allowing modern CPUs to process it via hardware acceleration at gigabits per second.

The Demise of RSA & The Rise of Ed25519

For two decades, engineers generated keys using `ssh-keygen -t rsa -b 4096`. RSA relies on Prime Factorization (multiplying two massive prime numbers together is easy; figuring out which primes were used to make the result is practically impossible). However, RSA keys are massive, slow, and theoretically vulnerable to future Quantum Computers (via Shor's Algorithm).

Modern environments mandate **Ed25519** (Elliptic Curve Cryptography). Instead of prime numbers, ECC uses the algebraic structure of elliptic curves over finite fields (e.g., $y^2 = x^3 + ax + b$).

- **Mathematical Superiority:** A tiny 256-bit Ed25519 key provides the exact same cryptographic strength as a massive 3072-bit RSA key.
- **Performance:** Because the key is so small, signature generation and verification are exponentially faster, drastically reducing SSH connection latency.

7.1 Key Management, ssh-agent, and the Forwarding Trap

The `ssh-agent` Daemon

Private keys should always be encrypted on your SSD with a strong passphrase. However, typing this passphrase 50 times a day is unacceptable. The `ssh-agent` is a background daemon. When you run `ssh-add`, you type your passphrase once. The agent decrypts the key and holds the raw bytes securely in your computer's RAM. It then creates a local UNIX socket (e.g., `/tmp/ssh-XXXXX/agent.sock`) and sets an environment variable (`SSH_AUTH_SOCK`) so the SSH client knows where to find it.

IRL Exploit: The ForwardAgent Vulnerability

To access a private production server (DB1), you must first SSH into a public Bastion host. Junior engineers often enable `ForwardAgent yes` in their config. This tells SSH to secretly map your laptop's local `ssh-agent` socket across the network onto the Bastion host's `/tmp/` directory.

The Catastrophe: A hacker has compromised the Bastion host and gained `root` access. They wait for you to log in.

1. They run `ls -l | grep agent` to find your forwarded socket file.
2. They run `export SSH_AUTH_SOCK=/tmp/ssh-1234/agent.sock` on their own terminal.
3. They run `ssh root@production_db`.

Because they hijacked the socket, the Bastion host asks *your laptop's RAM* to sign the cryptographic challenge. Your laptop silently complies. The hacker is instantly logged into the production database as you, bypassing all security, without ever stealing your actual private key file. **Never use ForwardAgent.**

7.2 Advanced `~/.ssh/config` Engineering

The `~/.ssh/config` file is a deterministic rules engine. When you execute `ssh web-prod`, the SSH client parses the file top-to-bottom. *The first matching rule wins*. This allows you to set host-specific overrides followed by global defaults (using `Host *` at the absolute bottom).

Connection Multiplexing (ControlMaster)

We established that the 5-phase SSH handshake is mathematically heavy, often taking 500ms-800ms. If you are running an Ansible playbook that connects to a server 100 times, you waste a full minute just doing Diffie-Hellman math.

ControlMaster solves this. When you SSH to a server for the first time, it executes the full cryptographic handshake and creates a background "Master" connection tied to a local socket file. If you open a second terminal and SSH to the same server, SSH detects the socket, bypasses the network handshake entirely, and simply asks the Master connection to spawn a new session. Connection time drops from 800ms to 5ms.

```
Host *  
    ControlMaster auto  
    ControlPath ~/.ssh/sockets/%r@%h:%p  
    ControlPersist 10m
```

NAT Firewalls & TCP KeepAlives

If you leave your SSH terminal idle, a corporate NAT firewall will eventually purge the inactive connection from its routing tables to save memory. When you return, your terminal is frozen. Setting `ServerAliveInterval 60` tells your local SSH client to send a tiny, encrypted application-layer ping every 60 seconds, tricking the NAT firewall into keeping the stateful connection open indefinitely.

The Modern Jump Host (`ProxyJump`)

To safely traverse a Bastion host without the lethal `ForwardAgent` vulnerability, modern OpenSSH uses `ProxyJump`.

```
Host bastion
  Hostname 1.2.3.4
  User admin

Host internal-db
  Hostname 10.0.5.99
  ProxyJump bastion
```

Mechanical Reality: When you type `ssh internal-db`, SSH connects to the bastion, but it *does not* start a shell there. It asks the Bastion to blindly forward raw TCP bytes to `10.0.5.99`. Your laptop then performs the Diffie-Hellman cryptographic handshake directly with the `internal-db`. The Bastion host only sees encrypted ciphertext passing through it. Even if root on the Bastion is compromised, they cannot read your traffic or hijack your keys.

7.3 SSH Daemon (`sshd`) OS Mechanics & Hardening

The Operating System Reality of `sshd`

How does a server actually receive an SSH connection? The master `sshd` process runs as `root`, permanently bound to TCP Port 22, listening for incoming SYN packets.

When a connection arrives, the master daemon immediately calls the `fork()` system call. The child process handles the connection. Crucially, OpenSSH uses **Privilege Separation**. The child process immediately drops its `root` privileges, switching to an unprivileged, restricted dummy user (like `sshd`) to process the highly complex, untrusted incoming network data (the Diffie-Hellman key exchange).

The Benefit of Privilege Separation

If a hacker discovers a Zero-Day Buffer Overflow exploit in the SSH protocol's cryptographic parsing engine, they can crash the child process and execute arbitrary code. However, because the child process dropped its privileges *before* parsing the math, the hacker is trapped as the unprivileged `sshd` user in an isolated sandbox, rather than instantly gaining full `root` control of the server.

Hardening `/etc/ssh/sshd_config`

A production server exposed to the public internet will endure thousands of brute-force dictionary attacks every hour. Hardening is not optional.

- `PasswordAuthentication no` : The most critical setting. Passwords can be brute-forced; a 256-bit Ed25519 key cannot.
- `PermitRootLogin prohibit-password` : Strictly prevents logging in directly as root via a password, forcing users to log in as a standard user and escalate via `sudo` (which leaves a detailed audit trail).
- `AllowUsers alice bob` : Creates an absolute whitelist. Even if "charlie" has a valid user account and a valid SSH key on the server, the daemon will instantly drop his connection at the TCP level.

The Ultimate Lockdown: `authorized_keys` Restrictions

When you place a public key in `~/.ssh/authorized_keys`, that key grants full shell access. But what if you are writing an automated CI/CD script that only needs to deploy code, or a cronjob that

only needs to trigger a backup? Giving a script full interactive shell access is a massive security risk.

IRL Implementation: Restricting Keys

You can prepend strict execution rules to a public key in the `authorized_keys` file. This applies the Principle of Least Privilege mathematically.

```
command="/opt/scripts/deploy.sh",no-pty,no-port-forwarding,no-X11-forwarding ssh-  
ed25519 AAAAC3...
```

The Mechanic: When the client authenticates using the matching private key, the `sshd` daemon intercepts the connection. It *refuses* to spawn a bash shell (enforced by `no-pty`). It ignores whatever command the client asked to run, and forcefully executes `/opt/scripts/deploy.sh` instead. If a hacker steals the private key from your Jenkins server, the key is useless to them—they can only trigger your deployment script; they cannot explore the server.

7.4 Tmux: The Terminal Multiplexer Architecture

Junior engineers view Tmux merely as a window manager to split their terminal screen. Staff engineers understand that Tmux is fundamentally a **background daemon and a pseudo-terminal (PTY) proxy** designed to decouple process execution from network stability.

The Client/Server OS Mechanics

When you connect to a server via SSH and execute a long-running database migration script, your SSH daemon (`sshd`) spawns a bash shell attached to your active TCP connection. If your laptop's Wi-Fi drops, the TCP connection times out. The Linux Kernel instantly sends a `SIGHUP` (Signal Hang Up) to your bash shell, which cascades and ruthlessly kills your migration script. The database is left in a corrupted state.

The Daemon Decoupling (Why Tmux Survives)

When you type `tmux` , it does not just draw a green bar on your screen. It executes a `fork()` system call, creating a hidden **Tmux Server Daemon** running in the background, bound to a UNIX socket (e.g., `/tmp/tmux-1000/default`).

Your visible terminal becomes a lightweight **Tmux Client** that connects to this socket. The background daemon requests virtual Pseudo-Terminals (PTYs) from the Kernel and runs your bash shells inside them.

If your Wi-Fi drops, the Kernel kills your SSH connection and the Tmux *Client*. But the Tmux *Server* is a daemon; it has no parent TTY. It ignores the `SIGHUP` . Your migration script continues executing perfectly in the background. When your Wi-Fi returns, you reconnect via SSH, type `tmux attach` , and the client simply re-binds to the background socket, instantly redrawing your screen exactly as you left it.

The Hierarchical Data Structure

Tmux organizes data in a strict hierarchy, heavily utilized when scripting automated workspaces:

- **Server:** The background daemon managing the UNIX socket.
- **Session:** A collection of windows. You can detach from a session and leave it running.
- **Window:** Equivalent to a graphical tab. Occupies the entire screen.
- **Pane:** A subdivided rectangle within a window, running its own distinct PTY and bash instance.

7.5 Advanced Tmux Scripting & Workflows

Because Tmux is driven entirely by a command-line interface, every action you take interactively (splitting panes, typing commands) can be scripted mathematically.

Pane Synchronization (The Broadcast Architecture)

During a high-severity incident, a microservice deployed across 4 different physical servers begins throwing HTTP 500 errors. You need to inspect the NGINX error logs on all 4 machines simultaneously.

IRL Diagnostic: Synchronized Keystroke Broadcasting

Instead of opening 4 terminal tabs and typing the SSH and `grep` commands 4 separate times, you utilize Tmux Pane Synchronization.

1. You split your screen into 4 panes.
2. You execute the Tmux command: `:set-window-option synchronize-panes on`
3. **The Mechanic:** Tmux intercepts your physical keyboard input from the client and multiplexes the byte stream, copying it directly into the standard input (`stdin`) file descriptors of all 4 PTYs simultaneously.
4. You type `ssh web-node-X` and execute `tail -f /var/log/nginx/error.log | grep CRITICAL`. The command runs across the cluster in perfect unison.

Programmatic Workspace Construction

You can use the `tmux send-keys` command to inject strings and the Enter key (`C-m` for Carriage Return) into background, detached sessions. This allows you to write bash scripts that construct complex diagnostic dashboards before you even attach to them.

7.6 Production Implementation: Automating Access

Implementation 1: The Staff-Level `~/.ssh/config`

This configuration demonstrates how to drastically reduce latency via multiplexing, bypass firewalls automatically with ProxyJump, and keep idle sessions alive natively without altering server-side daemon settings.

```
# 1. GLOBAL DEFAULTS (Applied to all connections)
Host *
    # Keep idle connections alive by sending a tiny encrypted ping every 60 seconds.
    ServerAliveInterval 60
    # If the ping fails 3 times, gracefully terminate the connection.
    ServerAliveCountMax 3
    # Enforce modern Elliptic Curve keys, disabling legacy RSA.
    PubkeyAcceptedAlgorithms +ssh-ed25519
    # Enable Socket Multiplexing to reduce KEX handshake latency.
    ControlMaster auto
    ControlPath ~/.ssh/sockets/%r@%h:%p
    # Keep the invisible master connection alive for 15 minutes after you exit.
    ControlPersist 15m

# 2. THE BASTION HOST (The Entry Point)
Host prod-bastion
    HostName bastion.production.internal.com
    User security-admin
    IdentityFile ~/.ssh/id_ed25519_bastion

# 3. INTERNAL CLUSTER (Seamless hopping)
# By typing 'ssh db-primary', SSH will automatically connect to prod-bastion first,
# establish a secure TCP tunnel, and execute the final handshake with db-primary.
# ForwardAgent is explicitly NOT used to preserve key security.
Host db-primary
    HostName 10.0.99.50
    User postgres-admin
    IdentityFile ~/.ssh/id_ed25519_db
    ProxyJump prod-bastion
```

Implementation 2: The Automated Incident Response Dashboard

A Bash script that utilizes the Tmux CLI to silently construct a 4-pane diagnostic workspace in the background. It connects to 4 web servers, executes a live log tail, synchronizes the panes for further manual commands, and finally attaches the user to the finished GUI.

```
#!/bin/bash
set -euo pipefail

SESSION_NAME="incident_dashboard"

# Check if the session already exists. If so, attach and exit.
if tmux has-session -t "$SESSION_NAME" 2>/dev/null; then
    echo "[+] Session exists. Attaching..."
    tmux attach-session -t "$SESSION_NAME"
    exit 0
fi

echo "[+] Constructing automated diagnostic workspace..."

# 1. Create a new detached (-d) session named "incident_dashboard"
tmux new-session -d -s "$SESSION_NAME" -n "WebCluster"

# 2. Split the window into 4 equal panes (2x2 grid)
# -h splits left/right, -v splits top/bottom
tmux split-window -h -t "$SESSION_NAME:0.0"
tmux split-window -v -t "$SESSION_NAME:0.0"
tmux split-window -v -t "$SESSION_NAME:0.2"

# 3. Inject commands into each specific pane using 'send-keys'
# C-m mimics pressing the 'Enter' key.
# We are routing traffic through the Bastion configured in our SSH config.
tmux send-keys -t "$SESSION_NAME:0.0" "ssh web-node-1" C-m
tmux send-keys -t "$SESSION_NAME:0.0" "tail -f /var/log/syslog" C-m

tmux send-keys -t "$SESSION_NAME:0.1" "ssh web-node-2" C-m
tmux send-keys -t "$SESSION_NAME:0.1" "tail -f /var/log/syslog" C-m

tmux send-keys -t "$SESSION_NAME:0.2" "ssh web-node-3" C-m
tmux send-keys -t "$SESSION_NAME:0.2" "tail -f /var/log/syslog" C-m

tmux send-keys -t "$SESSION_NAME:0.3" "ssh web-node-4" C-m
tmux send-keys -t "$SESSION_NAME:0.3" "tail -f /var/log/syslog" C-m

# 4. Synchronize the panes so any subsequent typing is broadcasted to all 4 servers
tmux set-window-option -t "$SESSION_NAME:0" synchronize-panes on

echo "[+] Workspace ready. Attaching terminal..."
# 5. Hand control over to the user
tmux attach-session -t "$SESSION_NAME"
```

7.7 Chapter Verification, Checklist & Master Quiz

Staff-Level Verification Validators

- ☐ I can explain why Diffie-Hellman Key Exchange prevents Man-in-the-Middle attackers from deriving the symmetric encryption key.
- ☐ I understand why Ed25519 (Elliptic Curve) keys provide mathematically superior security and lower latency compared to legacy 4096-bit RSA keys.
- ☐ I can architect an SSH connection via a Bastion host using `ProxyJump`, strictly avoiding the catastrophic risks of `ForwardAgent`.
- ☐ I know how to restrict an automated public key in `authorized_keys` to execute only a specific script, preventing interactive shell access.
- ☐ I can explain the OS-level mechanics of why a Tmux daemon survives a `SIGHUP` signal when a TCP connection terminates.
- ☐ I can programmatically construct a multi-pane Tmux workspace using `split-window` and `send-keys` within a bash script.

Comprehensive Examination

1. Mechanically, how does a hacker hijack your `ssh-agent` if they gain root access on a shared Bastion host where you utilized `ForwardAgent yes` ?

A) They copy your physical private key file from your laptop's SSD.

B) They intercept the Diffie-Hellman math and derive your private key.

C) They find your forwarded UNIX socket in `/tmp/`, set their `SSH_AUTH_SOCK` environment variable to point to it, and force your laptop's RAM to sign cryptographic challenges on their behalf.

2. Why does the SSH protocol transition from Asymmetric Cryptography to Symmetric Cryptography immediately after the handshake?

A) Asymmetric keys expire after 30 seconds.

B) Asymmetric math (like Elliptic Curves) is computationally expensive and too slow for high-speed bulk data transfer. Symmetric math (like AES) is hardware-accelerated and capable of gigabit throughput.

C) Symmetric cryptography is mathematically harder to crack than Asymmetric.

3. How does the `ControlMaster` directive reduce SSH connection latency from ~800ms down to ~5ms?

A) It bypasses the firewall entirely using UDP.

B) It caches the DNS resolution in your local `/etc/hosts` file.

C) It executes the heavy Diffie-Hellman cryptographic handshake once, binding it to a local UNIX socket. Subsequent connections reuse this established, encrypted TCP tunnel without renegotiating the math.

4. What exactly happens on the Linux filesystem when a Tmux client detaches from a session?

A) The processes inside the session are paused (SIGSTOP) and written to the swap partition.

B) The visible client process exits, but the background Tmux Server Daemon (which owns the Pseudo-Terminals and ignores SIGHUP) continues running flawlessly in memory, bound to its UNIX socket.

C) Tmux executes an `nohup` command on all active PIDs.

5. In `sshd_config`, what is the architectural benefit of OpenSSH Privilege Separation?

A) It forces users to log in with SSH keys instead of passwords.

B) When a connection arrives, the root daemon forks a child process that immediately drops root privileges. If a hacker exploits a bug in the protocol parsing math, they are trapped as an unprivileged user rather than gaining instant system control.

C) It prevents multiple users from logging into the same server simultaneously.

Systems Writing Prompts

Prompt 1: Defending the Bastion

A junior engineer on your team suggests using `ForwardAgent yes` to easily traverse the company's new Bastion host into the secure database subnet. Draft a technical memo explaining the catastrophic security flaws of Agent Forwarding, and provide the exact `~/.ssh/config` implementation using `ProxyJump` as the secure alternative.

Prompt 2: The Resilient Pipeline

You have a Python script that takes 6 hours to analyze a massive 500GB log file on a remote server. Explain the exact OS-level signaling mechanics of why running this script directly in a standard SSH session is dangerous, and detail how executing it inside Tmux protects the process from TCP timeouts.

CHAPTER 7 TECHNICAL ANSWER KEY

1. **(C)** A forwarded agent creates a local socket on the remote machine. Any user with `root` access to that remote machine can hijack the socket and impersonate your cryptographic identity seamlessly.
2. **(B)** Asymmetric math requires complex curve derivations or prime factorization, which is CPU-bound. Symmetric math uses simple, reversible byte-shifting (AES), allowing hardware acceleration.
3. **(C)** `ControlMaster` multiplexes multiple SSH sessions over a single, pre-authenticated TCP connection, skipping the 5-phase handshake for all subsequent connections.
4. **(B)** Tmux's architecture decouples the visible frontend (Client) from the executing backend (Server daemon). Detaching simply severs the local socket connection; the daemon ignores the disconnect.
5. **(B)** Privilege Separation isolates the complex, bug-prone network parsing logic into an unprivileged sandbox. Only after authentication succeeds does the daemon pass the file descriptors to a privileged process to spawn the user's shell.

Prompt 1 (ProxyJump vs ForwardAgent): Agent Forwarding exposes the decrypted private key signing mechanism to the Bastion's root user via a `/tmp/` socket. `ProxyJump` instructs the Bastion to blindly route encrypted TCP packets to the destination, meaning the Bastion never sees plaintext data and cannot hijack the authentication challenge.

Prompt 2 (SIGHUP Mechanics): If the TCP connection drops, the kernel sends a `SIGHUP` to the bash session, terminating all child processes (including the 6-hour script). Tmux runs as a background daemon without a controlling TTY parent, rendering it immune to `SIGHUP`. The script continues safely in the background until the user reconnects and reattaches.